Lab Assignment #2

Math 437 - Modern Data Analysis - Jason Soto

Introduction

In this lab, we will explore the family-wise error rate (FWER) and false discovery rate (FDR), and then we will do some nonparametric testing using a permutation-based t-test.

Good Practice

Remember that we want to include two chunks at the beginning of each of our lab notebooks: the first loads all the packages we will be using and the second loads all the data.

Here I've added some new code with the `here` package to specify exactly where inside my project folder this lab notebook is. Generally, if you're doing this, you want `here::i_am("relative path to file")` to be the very first line of your notebook.

```
here::i_am("Labs/Lab02.qmd")
```

```
here() starts at C:/Users/jsnsoto04/Desktop/School/437
```

```
# change the name/capitalization of your Labs folder if necessary

# Now we load all the packages we'll use in the lab
library(ISLR2)
```

To import a dataset you have downloaded (for example, datasets I give you on Canvas), first **move the dataset out of your Downloads folder** and into a folder set up specifically for data (ideally, a subfolder of your project).

If you followed the instructions to set up your file structure for Math 437 and create a project for your work, the example chunk below should import the data correctly (possibly you will have to change the folder name from "Data" to whatever you named it).

```r
alcohol <- readr::read_csv(here::here("Data/alcohol.csv"))
```

If it didn't work, then find the *Environment* panel and click *Import Dataset*. Select the right type of file (you typically want `From Text (readr)` for my csv files), click *Browse* and find the file, then copy the *Code Preview* section and overwrite the example chunk. Note that the code to import your dataset now includes your entire file structure, which means that other people won't be able to replicate your data import.

This lab assignment is worth a total of **12.5 points**.

Problem 1: FWER and FDR (4.5 pts total)

Part a (Code: 1.5 pts)

ISLR Labs 13.6.1, 13.6.2, and 13.6.3.

Solution

Remember that every time you see a new chunk in the lab, create a new chunk in the Quarto document and *manually type in* the code in the chunk, then run that code.

You do not need to `label` each chunk, but it may be helpful if you are working with R for the first time.

```r
set.seed(6)
x <- matrix(rnorm(10 * 100), 10, 100)
x[, 1:50] <- x[, 1:50] + 0.5
```

```r
t.test(x[, 1], mu = 0)
```

```
    One Sample t-test

data:  x[, 1]
t = 2.0841, df = 9, p-value = 0.06682
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 -0.05171076  1.26242719
sample estimates:
mean of x
0.6053582
```

```r
p.values <- rep(0, 100)
for (i in 1:100)
  p.values[i] <- t.test(x[, i], mu = 0)$p.value
decision <- rep("Do not reject H0", 100)
decision[p.values <= .05] <- "Reject H0"

table(decision, c(rep("H0 is False", 50), rep("H0 is True", 50)))
```

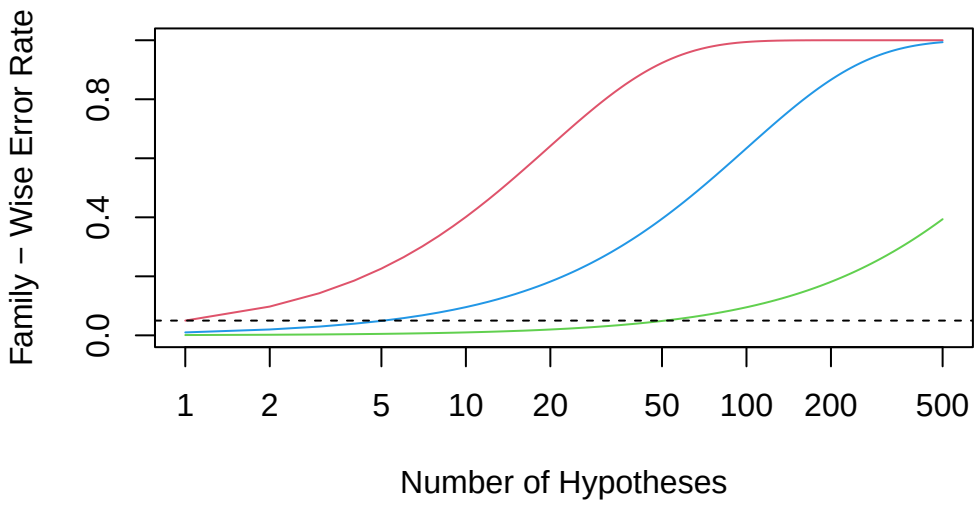
```
decision           H0 is False H0 is True
  Do not reject H0          40         47
  Reject H0                 10          3
```

```r
x <- matrix(rnorm(10 * 100), 10, 100)
x[, 1:50] <- x[, 1:50] + 1
for (i in 1:100)
  p.values[i] <- t.test(x[, i], mu = 0)$p.value
decision <- rep("Do not reject H0", 100)
decision[p.values <= .05] <- "Reject H0"
table(decision, c(rep("H0 is False", 50), rep("H0 is True", 50)))
```

```
decision           H0 is False H0 is True
  Do not reject H0           9         49
  Reject H0                 41          1
```

```r
m <- 1:500
fwe1 <- 1 - (1 - 0.05)^m
fwe2 <- 1 - (1 - 0.01)^m
fwe3 <- 1 - (1 - 0.001)^m
```

```r
par(mfrow = c(1, 1))
plot(m, fwe1, type = "l", log = "x", ylim = c(0, 1), col = 2, ylab = "Family - Wise Error Ra
lines(m, fwe2, col = 4)
lines(m, fwe3, col = 3)
abline(h = 0.05, lty = 2)
```




```r
library(ISLR2)
fund.mini <- Fund[, 1:5]
t.test(fund.mini[, 1], mu = 0)
```

```
    One Sample t-test

data:  fund.mini[, 1]
t = 2.8604, df = 49, p-value = 0.006202
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 0.8923397 5.1076603
sample estimates:
mean of x
        3
```

```r
fund.pvalue <- rep(0, 5)
for (i in 1:5)
  fund.pvalue[i] <- t.test(fund.mini[, i], mu = 0)$p.value
fund.pvalue
```

```
[1] 0.006202355 0.918271152 0.011600983 0.600539601 0.755781508
```

```r
p.adjust(fund.pvalue, method = "bonferroni")
```

```
[1] 0.03101178 1.00000000 0.05800491 1.00000000 1.00000000
```

```r
pmin(fund.pvalue * 5, 1)
```

```
[1] 0.03101178 1.00000000 0.05800491 1.00000000 1.00000000
```

```r
p.adjust(fund.pvalue, method = "holm")
```

```
[1] 0.03101178 1.00000000 0.04640393 1.00000000 1.00000000
```

```r
apply(fund.mini, 2, mean)
```

```
Manager1 Manager2 Manager3 Manager4 Manager5
     3.0     -0.1      2.8      0.5      0.3
```

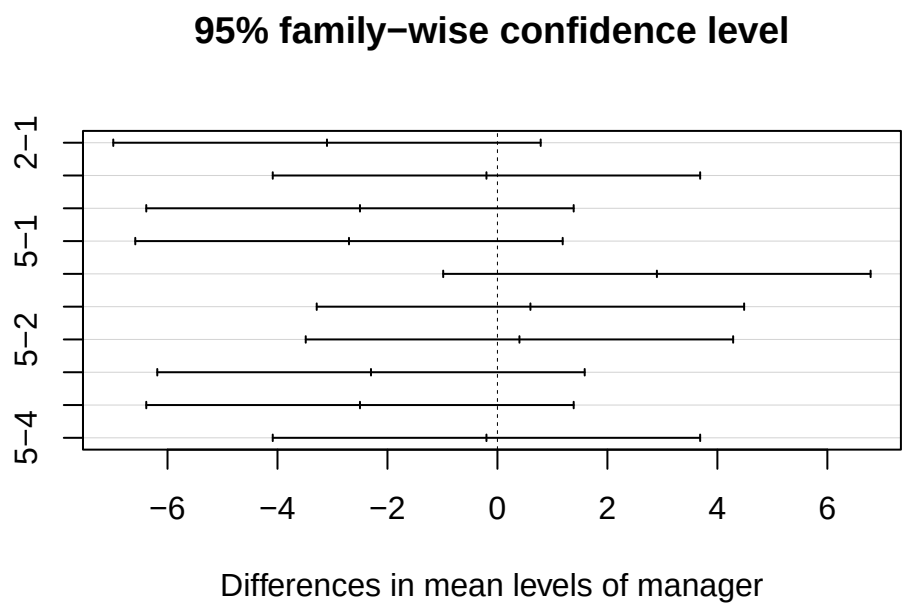
```r
t.test(fund.mini[, 1], fund.mini[, 2], paired = T)
```

```
    Paired t-test

data:  fund.mini[, 1] and fund.mini[, 2]
t = 2.128, df = 49, p-value = 0.03839
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.1725378 6.0274622
sample estimates:
mean difference
            3.1
```

```r
returns <- as.vector(as.matrix(fund.mini))
manager <- rep(c("1", "2", "3", "4", "5"), rep(50, 5))
a1 <- aov(returns ~ manager)
TukeyHSD(x = a1)
```

```
  Tukey multiple comparisons of means
    95% family-wise confidence level

Fit: aov(formula = returns ~ manager)

$manager
    diff        lwr       upr     p adj
2-1 -3.1 -6.9865435 0.7865435 0.1861585
3-1 -0.2 -4.0865435 3.6865435 0.9999095
4-1 -2.5 -6.3865435 1.3865435 0.3948292
5-1 -2.7 -6.5865435 1.1865435 0.3151702
3-2  2.9 -0.9865435 6.7865435 0.2452611
4-2  0.6 -3.2865435 4.4865435 0.9932010
5-2  0.4 -3.4865435 4.2865435 0.9985924
4-3 -2.3 -6.1865435 1.5865435 0.4819994
5-3 -2.5 -6.3865435 1.3865435 0.3948292
5-4 -0.2 -4.0865435 3.6865435 0.9999095
```

```r
plot(TukeyHSD(x = a1))
```




```r
fund.pvalues <- rep(0, 2000)
for (i in 1:2000)
  fund.pvalues[i] <- t.test(Fund[, i], mu = 0)$p.value
```

```r
q.values.BH <- p.adjust(fund.pvalues, method = "BH")
q.values.BH[1:10]
```

```
 [1] 0.08988921 0.99149100 0.12211561 0.92342997 0.95603587 0.07513802
 [7] 0.07670150 0.07513802 0.07513802 0.07513802
```

```r
sum(q.values.BH <= .1)
```

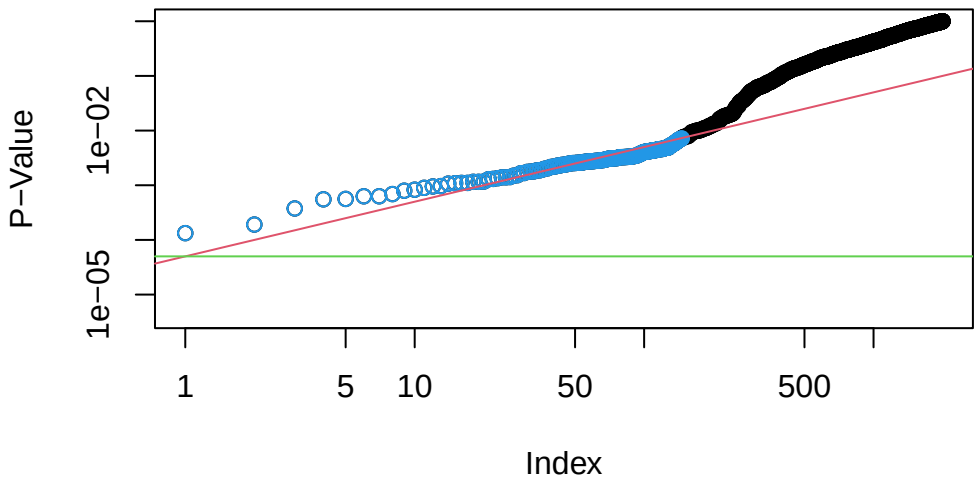
```
[1] 146
```

```r
sum(fund.pvalues <= (0.1 / 2000))
```

```
[1] 0
```

```r
ps <- sort(fund.pvalues)
m <- length(fund.pvalues)
q <- 0.1
wh.ps <- which(ps < q * (1:m) / m)
if (length(wh.ps) >0) {
  wh <- 1:max(wh.ps)
  } else {
    wh <- numeric(0)
}
```

```r
plot(ps, log = "xy", ylim = c(4e-6, 1), ylab = "P-Value", xlab = "Index", main = "")
points(wh, ps[wh], col = 4)
abline(a = 0, b = (q / m), col = 2, untf = TRUE)
abline(h = 0.1 / 2000, col = 3)
```



Part b (Computation and/or Explanation: 1 pt)

Using the output of Lab 13.6.1, estimate the power of this test to detect each of the two alternative hypotheses $\mu = 0.5$ and $\mu = 1$.

Solution

```
x <- matrix(rnorm(10 * 100), 10, 100)
x[, 1:50] <- x[, 1:50] + 1
for (i in 1:100)
  p.values[i] <- t.test(x[, i], mu = .5)$p.value
decision <- rep("Do not reject H0", 100)
decision[p.values <= .05] <- "Reject H0"
table(decision, c(rep("H0 is False", 50), rep("H0 is True", 50)))
```

```
decision           H0 is False H0 is True
  Do not reject H0          33         35
  Reject H0                 17         15
```

```r
x <- matrix(rnorm(10 * 100), 10, 100)
x[, 1:50] <- x[, 1:50] + 1
for (i in 1:100)
  p.values[i] <- t.test(x[, i], mu = 1)$p.value
decision <- rep("Do not reject H0", 100)
decision[p.values <= .05] <- "Reject H0"
table(decision, c(rep("H0 is False", 50), rep("H0 is True", 50)))
```

```
decision           H0 is False H0 is True
  Do not reject H0          48         10
  Reject H0                  2         40
```

Hence Power = 1-17/50 = .66 when mu = .5 and Power = 1-40/50 = .2 when mu = .5

Part c (Code: 1.5 pts)

Finish writing the `FDR_plot` function below by copying and pasting the code from the last two chunks of ISLR Lab 13.6.3. This function takes two arguments as inputs: `p.values`, a vector of p-values, and `q`, the desired False Discovery Rate. You will need to replace variables and hard-coded values in the code you copied with `p.values` and `q` so that the function runs.

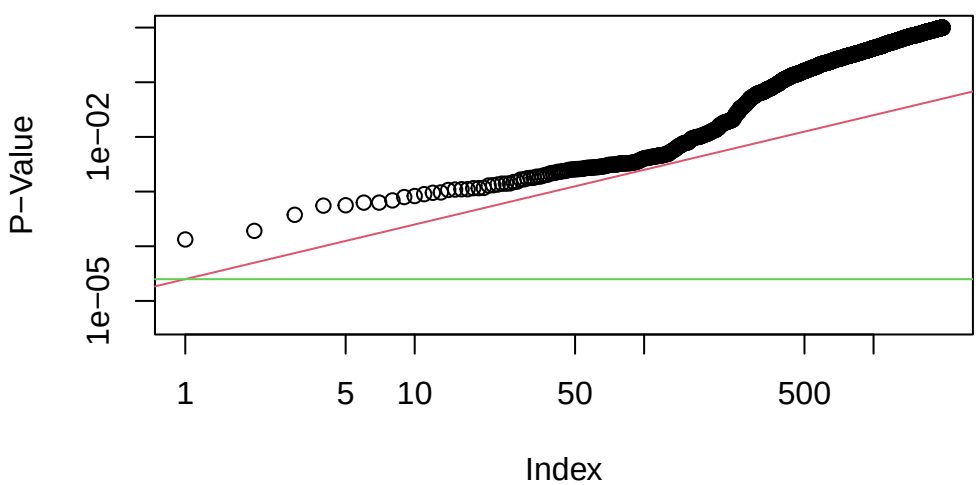
Once the function works correctly, call the function and produce graphs like those in Figure 13.6 from the outputs. (Assume that on the graphs, q and α are set to the same value.)

Solution

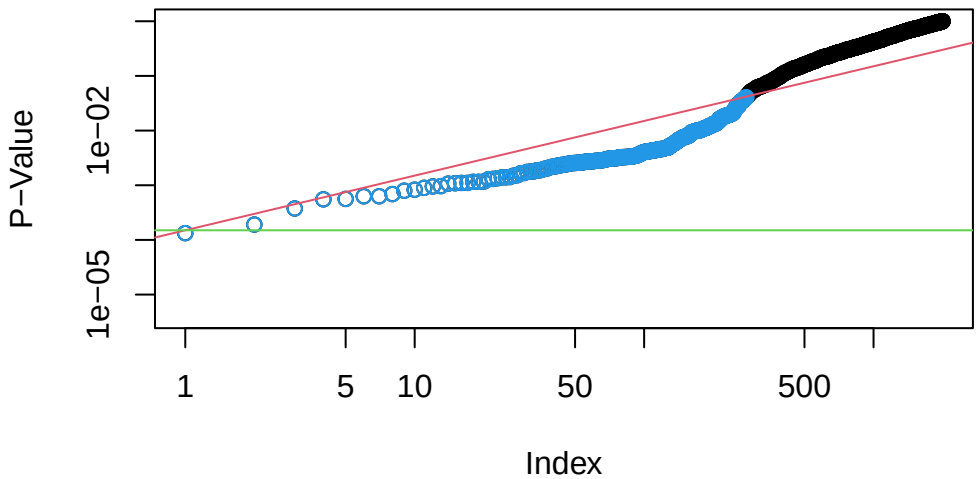
```r
FDR_plot <- function(p.values, q){

  # Copy code from the last two chunks of ISLR Lab 13.6.3, but replace variable names and ha

  # Since the main goal of this function is to produce a plot,
  # we "invisibly" return a vector with the significant p-values

ps <- sort(p.values)
m <- length(p.values)
wh.ps <- which(ps < q * (1:m) / m)
if (length(wh.ps) >0) {
  wh <- 1:max(wh.ps)
  } else {
```

```
    wh <- numeric(0)
  }

plot(ps, log = "xy", ylim = c(4e-6, 1), ylab = "P-Value", xlab = "Index", main = "")
points(wh, ps[wh], col = 4)
abline(a = 0, b = (q / m), col = 2, untf = TRUE)
abline(h = q / m, col = 3)

  invisible(wh.ps)
}
```

```
FDR_plot(fund.pvalues, .05)
```




```
FDR_plot(fund.pvalues, .3)
```



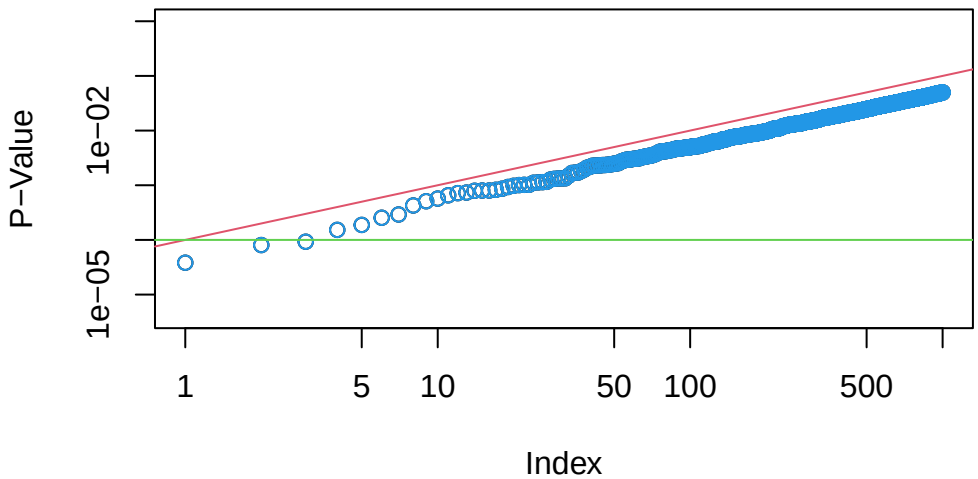
Part d (Code: 0.5 pts)

Apply your function from Part c on the simulated set of p-values in the chunk below, using $q = 0.1$, and produce a graph similar to those from Part b.

```
set.seed(12)
sim_pvalues <- runif(1000, min = 1e-5, max = 0.05001)
```

Solution

```
FDR_plot(sim_pvalues, .1)
```



Problem 2: The `p.adjust` Function (4 pts total)

Consider the following set of five p-values:

```
sim_pvalues2 <- c(0.02, 0.03, 0.06, 0.15, 0.25)
```

Part a (Code and Explanation: 1 pt)

Apply the `p.adjust` function with `method = "bonferroni"` to this set of p-values. Explain precisely what this method does. Pay special attention to what `p.adjust` does with the value `0.25`.

Solution

```
p.adjust(sim_pvalues2, method = "bonferroni")
```

```
[1] 0.10 0.15 0.30 0.75 1.00
```

What Bonferroni does is returns p values after being multiplied by the number of comparisons. In this case, 5 p-values, so each p-value is multiplied by 5, with .25 capping at 1.00 since p-values are probabilities.

Part b (Code and Explanation: 1 pt)

Apply the `p.adjust` function with `method = "holm"` to this set of p-values. Explain precisely what this method does. Pay special attention to the last two p-values.

Solution

```r
p.adjust(sim_pvalues2, method = "holm")
```

```
[1] 0.10 0.12 0.18 0.30 0.30
```

What Holm does is before a p-value is returned is similar to Bonferroni, except instead of multiplying by just the number of comparisons, it is multiplied by that said number + 1 - i, with i being the index of the order of the affected p-value. Though, the final value should be .25 following this, the reason it is .30 is because in this method, p-values must stay in order from small to big, hence, to retain validity, .25 must be adjusted to at least the same value as the p-value before, which is .30. This method is less conservative than the last and retains a greater power which means fewer Type II Errors.

Part c (Code and Explanation: 1 pt)

Apply the `p.adjust` function with `method = "BH"` to this set of p-values. Explain precisely what this method does. Pay special attention to the first two p-values.

Solution

```r
p.adjust(sim_pvalues2, method = "BH")
```

```
[1] 0.0750 0.0750 0.1000 0.1875 0.2500
```

What BH does is very similar to Holm, except it multiplies the p-value by the quantity, and divides by the placement of the p-value. Another difference is that .02 was sized down to be equivalent to .075. So when the adjusted p-value is bigger than the value to it's right, it is sized down to an equivalent. This method is less conservative than the last and retains a greater power which means fewer Type II Errors.

Part d (Explanation: 1 pt)

Why does it make more sense for R to adjust the p-values rather than the significance level?

Solution

It make more sense for R to adjust the p-values rather than the significance level because I would assume it makes p-values can be easily comparable with other datasets and stuff like that.

Nevermind, that's what I thought it was, it is solely because it knows what the p-value is and not the significance level.

Problem 3: Permutation Test for Comparing Two Means (4 pts total)

24 "alcohol-dependent" male patients at an alcohol treatment facility were randomly assigned to a traditional treatment program (Control) or a treatment program plus social skills training (SST). 23 patients faithfully reported their alcohol intake for a year (one patient in the SST group never reported). Their group assignment, and alcohol intake over the year (in centiliters of pure alcohol), is found in the *alcohol.csv* dataset on Canvas.

Using a permutation t-test, we wish to determine whether there is convincing evidence of a difference in population mean alcohol consumption between patients using the SST program and patients using a traditional treatment program.

Part a (Explanation: 1 pt)

Specify the null and (two-sided NHST) alternative hypothesis using both words and math.

Solution

The null hypothesis would be that there is no difference in effectiveness between the two programs. The alternative is that a difference between them exists.

H0: mu1 = mu2 & Ha: mu1 /= mu2

Part b (Code: 1.5 pts)

Construct a graph showing the permutation null distribution of t-statistics. Use 999 random permutation resamples. Include a vertical line at the observed value.

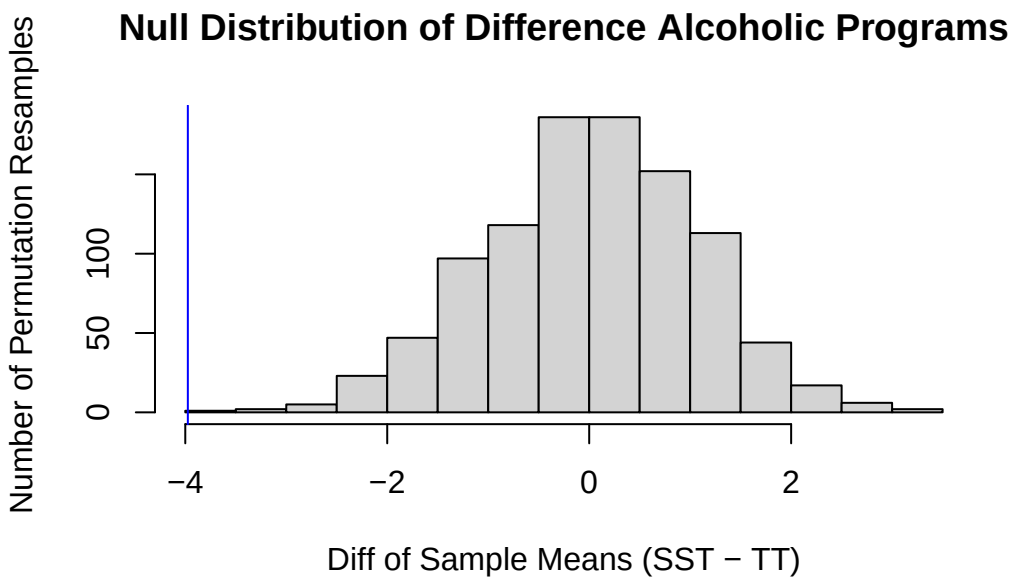
Solution

```r
set.seed(67)

B <- 999
sim_t <- numeric(B) # 999 zeroes

for (i in seq_along(sim_t)){
  resample <- alcohol                 # probably inefficient, but doing it here to illustrate
  resample$Intake <- sample(resample$Intake)    # randomly sample from resample$Intake withou

  SST <- which(resample$Group == "SST")         # Returns index numbers for each group
  CTRL <- which(resample$Group == "Control")


  sim_t[i] <- t.test(resample$Intake[SST], resample$Intake[CTRL])$statistic
}
```

```r
  SST <- which(alcohol$Group == "SST")
  CTRL <- which(alcohol$Group == "Control")

obs_t <- t.test(alcohol$Intake[SST], alcohol$Intake[CTRL])$statistic
```

```r
null_dist <- sim_t
```

```r
hist(
  null_dist,
  main = "Null Distribution of Difference Alcoholic Programs",
  xlab = "Diff of Sample Means (SST - TT)",
  ylab = "Number of Permutation Resamples"
)
abline(v = obs_t, col = "blue")
```



Note: R runs properly, render does not load the line correctly, should be between -2 and -1

Part c (Code: 0.5 pts)

Using the permutation null distribution, obtain the p-value for the two-sided test.

Solution

```
p_value <- mean(abs(sim_t) >= abs(obs_t))
p_value
```

```
[1] 0
```

Note: R runs properly, render does not load p-value, I got, [1] 0.19219219

Part d (Explanation: 1 pt)

Is there is convincing evidence of a difference in population mean alcohol consumption between patients using the SST program and patients using a traditional treatment program? Why or why not?

Solution

There is no convincing evidence in the difference because my found (maybe wrong) p-value is over than .05. Though, we do see a skew to the left in the bar graph, implying that SST is the superior treatment.